

LAB REPORT

Experiment 5: Docker Fundamentals

Software Engineering Laboratory | Two-container Python HTTP application

Student Danial Farahani

Student number 97105725

1. Objective

The aim was to deploy an HTTP server and client as separate Docker containers, configure their communication with Docker Compose, and demonstrate Dockerfiles, internal service networking, and host-to-container port forwarding. The implementation follows the two-program exercise in the Experiment 5 handout (pages 1–4).

2. Environment and application

Component	Observed or used configuration
Host and backend	Windows; Docker Desktop using Linux containers with WSL 2
Docker / Compose	Docker 29.6.1; Docker Compose 5.3.0
Image base	python:3.10-alpine for both images
Application	server.py listens on TCP 80; client.py sends five HTTP GET requests
Container discovery	SERVER_HOST=my-server; requests to http://my-server:80
Host exposure	TCP localhost:8300 mapped to server container port 80

Application flow

The server returns a plain-text greeting for an HTTP GET request. The client reads SERVER_HOST from its environment and sends five requests, three seconds apart. Compose creates the shared application network, so the client resolves the server by its service name; the host accesses the same server through the published port.

The source listings in the handout are clipped at their right edges. The starter code reconstructed the incomplete lines while preserving the behavior described in the assignment. The source was then tested using the observed Docker run, rather than assumed correct from inspection alone.

3. Implementation

3.1 Dockerfiles

Each service uses its own Dockerfile. FROM selects the Python Alpine base, WORKDIR sets /app, COPY puts the corresponding script in the image, and CMD starts Python in unbuffered mode so log output appears promptly. The server additionally documents its listening port with EXPOSE 80; actual publication is configured in Compose.

Server Dockerfile	Client Dockerfile
FROM python:3.10-alpine	FROM python:3.10-alpine
WORKDIR /app	WORKDIR /app
COPY server.py .	COPY client.py .
EXPOSE 80	(No port exposed by client)
CMD ["python", "-u", "server.py"]	CMD ["python", "-u", "client.py"]

3.2 Docker Compose

```
services:
  my-server:
    build:
      context: ./server
    ports:
      - "8300:80"
  my-client:
    build:
      context: ./client
    environment:
      SERVER_HOST: my-server
    depends_on:
      - my-server
```

The two build contexts select the server and client folders. The 8300:80 mapping publishes port 80 of the server container on the Windows host at port 8300. SERVER_HOST supplies the Compose service name to the client. depends_on controls startup ordering, but does not guarantee HTTP readiness; the client code catches connection errors.

3.3 Commands used

```
docker compose config --quiet
docker compose build
docker compose up -d
docker compose ps -a
curl.exe -i http://localhost:8300
docker compose logs --no-color my-client
docker compose exec my-server sh
```

Inside the server shell, pwd, ls -la, and exit were executed. The original host-port mapping (8000:80) failed because Windows excluded TCP ports 7967–8066. Changing only the host side to 8300 resolved the bind error without changing the scripts or internal port.

4. Execution evidence

The following images are genuine screenshots supplied from the local VS Code / PowerShell session. They document results obtained on the student's machine; the report author did not independently execute Docker in that environment.

4.1 Image build

docker compose build completed successfully and produced both exp5-my-server and exp5-my-client images.

```

PS C:\Users\Farah\Desktop\summer\SeLab\SELabHomeworks\exp5>
>> docker compose build
#11 [my-client 2/3] WORKDIR /app
#11 DONE 0.1s

#12 [my-server 3/3] COPY server.py .
#12 DONE 0.1s

#13 [my-client 3/3] COPY client.py .
#13 DONE 0.1s

#14 [my-client] exporting to image
#14 exporting layers
#14 exporting layers 0.4s done
#14 exporting manifest sha256:ffa06c0bc743500eb6352f5d1e4bef90ea115a894c6b6c53c4848d7bd458c69 0.1s done
#14 exporting config sha256:251f3d900a1251854e3b95d7674921f3e3643cd15d85b060e1c2b5b4503ef092
#14 exporting config sha256:251f3d900a1251854e3b95d7674921f3e3643cd15d85b060e1c2b5b4503ef092 0.1s done
#14 exporting attestation manifest sha256:79c7b30b1a87288fa18cbccc6cebd47affa139c4f77458d87d192ad878eb9f1
#14 exporting manifest list sha256:fa5514f11c40fae0a0ce4cd0f67add92557153a4434aae99a49740b0ec387e2e 0.1s done
#14 naming to docker.io/library/exp5-my-client:latest
#14 unpacking to docker.io/library/exp5-my-client:latest 0.1s done
#14 DONE 0.8s

#15 [my-server] exporting to image
#15 exporting layers 0.4s done
#15 exporting manifest sha256:f158165e2441d2f86d646d3affbe665ee74799ed016d8e476d46f606da26fb6d 0.1s done
#15 exporting config sha256:f62d32b86bde13ed16286e408efc5e7e5905aa3515df2ae614cc06fd1fe72c21 0.1s done
#15 exporting attestation manifest sha256:6182936c1cc8c8591e2f9d865a6624019b2224ca64a200d92067b5d9ac0b8fe 0.1s done
#15 exporting manifest list sha256:039571f200191475bc3752ea2b543504ff088287ca89b6a09cd504dc5f84f314 0.1s done
#15 naming to docker.io/library/exp5-my-server:latest 0.0s done
#15 unpacking to docker.io/library/exp5-my-server:latest 0.1s done
#15 DONE 0.9s

#16 [my-server] resolving provenance for metadata file
#16 DONE 0.0s

#17 [my-client] resolving provenance for metadata file
#17 DONE 0.0s
[+] build 2/2
  Image exp5-my-server Built
  Image exp5-my-client Built
PS C:\Users\Farah\Desktop\summer\SeLab\SELabHomeworks\exp5>
  
```

Figure 1. Completed build, including the two successful image results.

4.2 Deployment

With host port 8300 selected, docker compose up -d started both services. The displayed port binding is 8300 -> 80/tcp.

```

PS C:\Users\Farah\Desktop\summer\SeLab\SELabHomeworks\exp5>
>> docker compose up -d
[+] up 2/2
  Container exp5-my-server-1 Started
  Container exp5-my-client-1 Started
PS C:\Users\Farah\Desktop\summer\SeLab\SELabHomeworks\exp5> docker compose ps -a
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
exp5-my-client-1    exp5-my-client      "python -u client.py"  my-client  6 minutes ago  Up 10 seconds
exp5-my-server-1    exp5-my-server      "python -u server.py"  my-server  11 seconds ago  Up 10 seconds  0.0.0.0:8300->80/tcp, [::]:8300->80/tcp
PS C:\Users\Farah\Desktop\summer\SeLab\SELabHomeworks\exp5>
  
```

Figure 2. Compose startup and initial container status.

4.3 Communication and container inspection

The host request returned HTTP/1.0 200 OK and the server greeting. The client logs show five successful responses from http://my-server:80. In the later status check, the client exited with code 0 while the server was still running.

```
PS C:\Users\farah\Desktop\summer\SeLab\SELabHomeworks\exp5>
>> docker compose up -d
[+] up 2/2
  ✓ Container exp5-my-server-1 Started 0.4s
  ✓ Container exp5-my-client-1 Started 0.2s
PS C:\Users\farah\Desktop\summer\SeLab\SELabHomeworks\exp5> docker compose ps -a
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
exp5-my-client-1    exp5-my-client      "python -u client.py"  my-client  6 minutes ago  Up 10 seconds
exp5-my-server-1    exp5-my-server      "python -u server.py"  my-server  11 seconds ago Up 10 seconds  0.0.0.0:8300->80/tcp, [::]:8300->80/tcp
PS C:\Users\farah\Desktop\summer\SeLab\SELabHomeworks\exp5> curl.exe -i http://localhost:8300
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.10.21
Date: Tue, 22 Sep 2026 11:07:10 GMT
Content-type: text/plain; charset=utf-8

Hello! The answer was sent from the Docker server container.
PS C:\Users\farah\Desktop\summer\SeLab\SELabHomeworks\exp5> docker compose logs --no-color my-client
my-client-1 | The client started. Attempting to connect to the server at address: http://my-server:80
my-client-1 | [Request 1] Response received from server: Hello! The answer was sent from the Docker server container.
my-client-1 | [Request 2] Response received from server: Hello! The answer was sent from the Docker server container.
my-client-1 | [Request 3] Response received from server: Hello! The answer was sent from the Docker server container.
my-client-1 | [Request 4] Response received from server: Hello! The answer was sent from the Docker server container.
my-client-1 | [Request 5] Response received from server: Hello! The answer was sent from the Docker server container.
PS C:\Users\farah\Desktop\summer\SeLab\SELabHomeworks\exp5> ^C
PS C:\Users\farah\Desktop\summer\SeLab\SELabHomeworks\exp5> docker compose ps -a
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
exp5-my-client-1    exp5-my-client      "python -u client.py"  my-client  21 minutes ago Exited (0) 15 minutes ago
exp5-my-server-1    exp5-my-server      "python -u server.py"  my-server  15 minutes ago Up 15 minutes  0.0.0.0:8300->80/tcp, [::]:8300->80/tcp
PS C:\Users\farah\Desktop\summer\SeLab\SELabHomeworks\exp5>
```

Figure 3. HTTP response, all five client requests, and final container status.

The server shell reported /app as its working directory. The ls -la output includes server.py, confirming the script was copied into the image and is accessible inside the running container.

```
PS C:\Users\farah\Desktop\summer\SeLab\SELabHomeworks\exp5>
>> docker compose exec my-server sh
/app # pwd
/app
/app # ls -la
total 12
drwxr-xr-x  1 root    root      4096 Sep 22 10:44 .
drwxr-xr-x  1 root    root      4096 Sep 22 10:52 ..
-rwxr-xr-x  1 root    root       758 Sep 21 14:48 server.py
/app # exit
PS C:\Users\farah\Desktop\summer\SeLab\SELabHomeworks\exp5>
```

Figure 4. Server-container shell showing pwd, ls -la, and exit.

5. Answers to the handout questions

Q1. What does docker-compose.yml do, and when is it used instead of docker run?

docker-compose.yml declares a multi-container application: services, image builds, networking, environment variables, published ports, and dependencies. docker compose up creates and starts the configured services together. A single docker run command is adequate for an individual container; Compose is convenient when related containers must be configured and managed together, as with this server and client.

Q2. What is Kubernetes used for, and how does it relate to Docker?

Kubernetes orchestrates containerized applications across machines, handling deployment, scaling, networking, and recovery. Docker can build container images that Kubernetes runs using a compatible container runtime; Kubernetes does not require Docker Engine as its runtime.

Q3. Explain Docker Image, Container, and Volume.

Image: A reusable, read-only template containing the application, runtime, and dependencies; the Dockerfiles build two images in this experiment.

Container: A runnable instance of an image with its own isolated processes and filesystem view; my-server and my-client are separate containers.

Volume: Docker-managed persistent storage that can outlive a container and be mounted into containers. This exercise does not need a volume because it stores no persistent application data.

6. AI assistance disclosure

Tool/model: OpenAI ChatGPT was used to plan, scaffold, troubleshoot, organize evidence, and draft this report. The report was prepared using GPT-5.6 Sol; the exact model configuration for all earlier chat steps was not independently recorded.

Representative user requests included: “read and construct our roadmap for doing it”; “give me a zip file containing initial stage of exp5”; “what is this error”; and “give these screen shots you told me in a zip.”

Outputs and benefits: AI proposed the directory layout, reconstructed clipped Python source lines, prepared Docker/Compose configuration, explained the Windows port-exclusion error, and structured the report.

Limitations: PDF text was partially clipped; suggested code needed actual execution; and the AI did not have direct access to the student’s Docker daemon. The screenshots and terminal results were provided by the student and used as the evidence of execution.

7. Conclusion

Two images were built and the corresponding containers were deployed with Compose. The host received HTTP 200 from localhost:8300, the client obtained five server responses, and the server filesystem was inspected successfully. The host-port change addressed a Windows reservation while retaining port 80 inside the container. The teaching assistant waived the video requirement; documentation and the public repository link remain the submission materials.

Source: Software Engineering Lab, Experiment 5 handout, pages 1–4; local execution screenshots in evidence/ (Figures 1–4).